Algorithms and Data Structures

09.02.2022

Exercise 1 (4.0 points)
Given the following C recursive function:

```
void f (int v[], int l, int r) {
  int i, c;
  if (l >= r) {
    return;
  }
  c = (r+l)/2;
  f (v, l, c);
  f (v, c+1, r);
  return;
}
```

Write the recurrence equation describing the algorithm. Analytically, compute its asymptotic complexity in terms of number of operations. Please, report all computational steps required to reach the result and the final asymptotic complexity.

Solution
Please, see theacher's overhead for a complete analysis of the topic.

Exercise 2 (1.0 points)
Given the following array of integer values, perform the first step of quicksort to sort the array in ascending order, thus from the initial array generate the right and the left partitions.

```
 4  8  1 10  6  5  2  7  9  3
```

Report 3 integer values: The pivot selected on the original array, the pivot you would select on the left partition generated from the original array, and the pivot you would select on the right partition generated (again) from the original array. No other symbols must be included in the response. This is an example of response format: 13 1 10

Solution

```
### Quick sort
Solution format 1:_____________________________
Initial array:  4  8  1 10  6  5  2  7  9  3
_______________________________________________
Re l= 0;r= 1:   2  1
Re l= 0;r=-1:
Re l= 0;r=-1:      2
Re l= 0;r= 1:           10  6  5  4  7  9  8
Re l= 3;r= 6:            7  6  5  4
Re l= 3;r= 2:
Re l= 3;r= 2:               6  5  7
Re l= 4;r= 5:               6  5
Re l= 4;r= 3:
Re l= 4;r= 3:                  6
Re l= 4;r= 5:
Re l= 3;r= 6:                            9 10
Re l= 8;r= 8:                            9
Re l= 8;r= 8:_____________________________________
Solution format 2:_____________________________
Input array:   [0]4 [1]8 [2]1 [3]10 [4]6 [5]5 [6]2 [7]7 [8]9 [9]3
Pivot:         [9]3
Swaps:         [0<->6]4<->2  - [1<->2]8<->1  - [2<->9]8<->3
Output array:  [0]2 [1]1  - [2]3  - [3]10 [4]6 [5]5 [6]4 [7]7 [8]9 [9]8
_______________________________________________
Input array:   [0]2 [1]1  - [3]10 [4]6 [5]5 [6]4 [7]7 [8]9 [9]8
Pivot:         [1]1  - [9]8
Swaps:         [0<->1]2<->1  - [3<->7]10<->7  - [7<->9]10<->8
Output array:   - [0]1  - [1]2  - [3]7 [4]6 [5]5 [6]4  - [7]8  - [8]9 [9]10
_______________________________________________
Input array:   [3]7 [4]6 [5]5 [6]4  - [8]9 [9]10
Pivot:         [6]4  - [9]10
Swaps:         [3<->6]7<->4  - [9<->9]10<->10
Output array:   - [3]4  - [4]6 [5]5 [6]7  - [8]9  - [9]10  -
_______________________________________________
Input array:   [4]6 [5]5 [6]7
Pivot:         [6]7
Swaps:         [6<->6]7<->7
Output array:  [4]6 [5]5  - [6]7  -
_______________________________________________
Input array:   [4]6 [5]5
Pivot:         [5]5
Swaps:         [4<->5]6<->5
Output array:   - [4]5  - [5]6
_______________________________________________
```

Response
3 1 8

Exercise 3 (1.0 points)
A BST contains integer values included in the range 1-1000. Suppose we are looking for the value 0367 is such a BST. Consider the following sequences of values generated during a search.

```
594 599 843 604 827 790 673 646 658 667 669 672 670 671
749 265 742 721 368 633 697 644 670 683 669 677 676 671
```

Indicate which sequences (numbered as 1 and 2) are correct. Please, report the numbers indicating the correct sequences. These numbers must be separated by a single space and reported in ascending order. No other symbols must be included in the response. This is an example of the response format (when both sequences are correct): 1 2.

Solution

```
### BST verify sequence
Searching: 671
Sequence  1: 594 599 843 604 827 790 673 646 658 667 669 672 670 671
             594 599 843 604 827 790 673 646 658 667 669 672 670 671 Sequence OK.
Sequence  2: 749 265 742 721 368 633 697 644 670 683 669 677 676 671
             749 265 742 721 368 633 697 644 670 Error: 669<670
```

Response
1

Exercise 4 (1.0 points)
Insert the following sequence of keys into an initially empty hash table. The hash table has a size equal to `M=19`. Insertions occur character by character using open addressing with quadratic probing (with $c_1 = 1$ and $c_2 = 1$). Each character is identified by its index in the English alphabet (i.e., `A=1`, ..., `Z=26`). Equal letters are identified by a different subscript (i.e., `A` and `A` become `A1` and `A2`).

```
Z O R R O
```

Indicate in which elements are placed the last two letters of the sequence, i.e., `R` and `O`, in this order. Please, report your response as a sequence of integer values separated by one single space. No other symbols must be included in the response. This is an example of the response format: 3 4

Solution

```
### hash-table
Character keys
Hash table size = 19
Number of keys  =  5
Quadratic probing: c1=1.000000, c2=1.000000.
Key:  Z O R R O
Key=Z k=026 Final=07
Key=O k=015 Final=15
Key=R k=018 Final=18
Key=R k=018 Coll.=18 Final=01
Key=O k=015 Coll.=15 Final=17
00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18
-  R  -  -  -  -  -  Z  -  -  -  -  -  -  -  O  -  O  R
```

Response
1 17

Exercise 5 (1.0 points)
Suppose to have an initially empty priority queue implemented with a minimum heap. Consider the following sequence of integers and "*" characters, where each integer corresponds to one insertion into the priority queue and each character "*" corresponds to one extraction.

```
19 11  2  1  5  3 *  *
```

Report the sequence of values as they are stored in the array representing the priority queue at the end of the entire process. Please, show the entire content of the array as a sequence of integer values separated by a single space. No other symbols must be included in the response. This is an example of the response: 0 3 2 6 8 etc.

Solution

```
### Heap insertion and extraction
Max-Heap.
Insert keys: 19 11  2  1  5  3 *  *
Insert   19: 19
Insert   11: 19 11
Insert    2: 19 11  2
Insert    1: 19 11  2  1
Insert    5: 19 11  2  1  5
Insert    3: 19 11  3  1  5  2
Extract  19: 11  5  3  1  2
Extract  11:  5  2  3  1
```

Response
5 2 3 1

Exercise 6 (1.0 points)
The following capital letters are given with their absolute frequency.

```
A:9 B:14 C:3 D:6 E:7 F:5
```

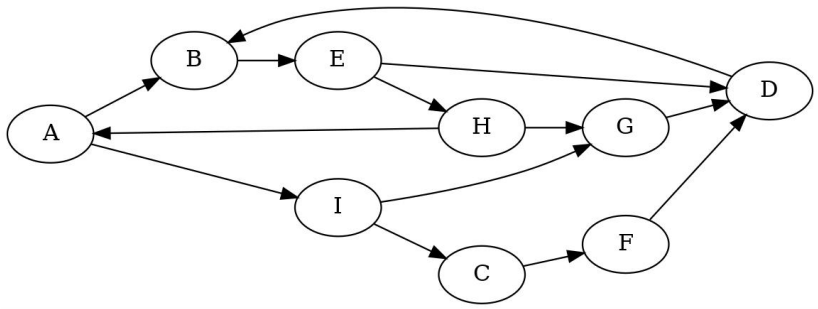
Find an optimal Huffman code for all symbols in the set using a greedy algorithm. Indicate the maximum number of bits that must be used to represent the symbol/symbols with the lowest frequency and the number of symbols that can be encoded with that same maximum number of bits. For example, if 3 letters must be represented with 5 bits (and all others with less than 5 bits) report as a response: 5 3. No other symbols must be included in the response.

Solution

```
### Huffman
Huffman:
Number of codes:  6
A:9 B:14 C:3 D:6 E:7 F:5
Merging: {C}(3)[-1][-1] + {F}(5)[-1][-1] = {CF}(8)[0][1]
Merging: {D}(6)[-1][-1] + {E}(7)[-1][-1] = {DE}(13)[2][3]
Merging: {CF}(8)[0][1] + {A}(9)[-1][-1] = {CFA}(17)[4][5]
Merging: {DE}(13)[2][3] + {B}(14)[-1][-1] = {DEB}(27)[6][7]
Merging: {CFA}(17)[4][5] + {DEB}(27)[6][7] = {CFADEB}(44)[8][9]
C 000
F 001
A 01
D 100
E 101
B 11
```

Response
3 4

Exercise 7 (1.0 points)
Visit the following graph in depth-first, starting at node A. Label each edge as tree (T), back (B), forward (F), and cross (C). When necessary, consider nodes and edges in alphabetic order.



Report the label of the edges DB, CF, and IG in this order. Please, indicate the edge type of these 3 arcs with a single letter, i.e., T, F, B, C, separated by single spaces. No other symbols must be included in the response. This is an example of the response: F T B

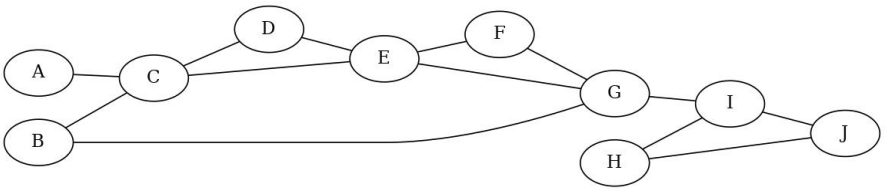
Solution

```
Directed matrix ... stated as such.
UN-weighted matrix ...stated as such.
Vertex list: [ 0] A [ 1] B [ 2] C [ 3] D [ 4] E [ 5] F [ 6] G [ 7] H [ 8] I
Adjacency Matrix:
 0 1 0 0 0 0 0 0 1
```

```
 0 0 0 0 1 0 0 0 0
 0 0 0 0 0 1 0 0 0
 0 1 0 0 0 0 0 0 0
 0 0 0 1 0 0 0 1 0
 0 0 0 1 0 0 0 0 0
 0 0 0 1 0 0 0 0 0
 1 0 0 0 0 0 1 0 0
 0 0 1 0 0 0 1 0 0
### Graph visit
BFS:
Node=[ 0] A Discovery_time= 0 Predecessor=[-1]
Node=[ 1] B Discovery_time= 1 Predecessor=[ 0]
Node=[ 2] C Discovery_time= 2 Predecessor=[ 8]
Node=[ 3] D Discovery_time= 3 Predecessor=[ 4]
Node=[ 4] E Discovery_time= 2 Predecessor=[ 1]
Node=[ 5] F Discovery_time= 3 Predecessor=[ 2]
Node=[ 6] G Discovery_time= 2 Predecessor=[ 8]
Node=[ 7] H Discovery_time= 3 Predecessor=[ 4]
Node=[ 8] I Discovery_time= 1 Predecessor=[ 0]
DFS:
List of edges:
[ 0] A -> [ 1] B : (T) Tree
[ 1] B -> [ 4] E : (T) Tree
[ 4] E -> [ 3] D : (T) Tree
[ 3] D -> [ 1] B : (B) Back
[ 4] E -> [ 7] H : (T) Tree
[ 7] H -> [ 0] A : (B) Back
[ 7] H -> [ 6] G : (T) Tree
[ 6] G -> [ 3] D : (C) Cross
[ 0] A -> [ 8] I : (T) Tree
[ 8] I -> [ 2] C : (T) Tree
[ 2] C -> [ 5] F : (T) Tree
[ 5] F -> [ 3] D : (C) Cross
[ 8] I -> [ 6] G : (C) Cross
List of vertices:
[ 0] A: dist_time= 1 endp_time=18 pred=[-1] -
[ 1] B: dist_time= 2 endp_time=11 pred=[ 0] A
[ 2] C: dist_time=13 endp_time=16 pred=[ 8] I
[ 3] D: dist_time= 4 endp_time= 5 pred=[ 4] E
[ 4] E: dist_time= 3 endp_time=10 pred=[ 1] B
[ 5] F: dist_time=14 endp_time=15 pred=[ 2] C
[ 6] G: dist_time= 7 endp_time= 8 pred=[ 7] H
[ 7] H: dist_time= 6 endp_time= 9 pred=[ 4] E
[ 8] I: dist_time=12 endp_time=17 pred=[ 0] A
Reachable nodes from vertex [ 0] A:
[ 0] A - [ 1] B - [ 2] C - [ 3] D - [ 4] E - [ 5] F - [ 6] G - [ 7] H - [ 8] I -
UN-reachable nodes from vertex [ 0] A:
none
```

Response
B T C

Exercise 8 (1.0 points)
Given the following graph find all bridges. If necessary, consider nodes and edges in alphabetical order.



Display all bridges. Please, report only the list of edges separated by a single space. The edges must be specified in alphabetic order and within each edge the two vertices must be specified in alphabetic order (i.e., AC and not CA). No other symbols must be included in the response. This is an example of the response: AB AZ BC etc.

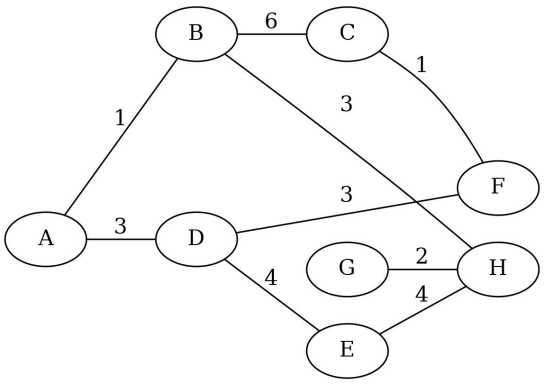
Solution

```
UN-directed matrix ... stated as such.
UN-weighted matrix ...stated as such.
Vertex list: [ 0] A [ 1] B [ 2] C [ 3] D [ 4] E [ 5] F [ 6] G [ 7] H [ 8] I [ 9] J
Adjacency Matrix:
 0 0 1 0 0 0 0 0 0 0
 0 0 1 0 0 0 1 0 0 0
 1 1 0 1 1 0 0 0 0 0
 0 0 1 0 1 0 0 0 0 0
 0 0 1 1 0 1 1 0 0 0
 0 0 0 0 1 0 1 0 0 0
 0 1 0 0 1 1 0 0 1 0
 0 0 0 0 0 0 0 0 1 1
 0 0 0 0 0 0 1 1 0 1
 0 0 0 0 0 0 0 1 1 0
```

```
### Graph Articulation Points and Bridges
Bridge: Edge [ 6]  G - [ 8]  I
Bridge: Edge [ 0]  A - [ 2]  C
Articulation point: Vertex [ 2]  C
Articulation point: Vertex [ 6]  G
Articulation point: Vertex [ 8]  I
```

Response
AC GI

Exercise 9 (1.0 points)
Given the following undirected and weighted graph find a minimum spanning tree using Prim algorithm. Start from vertex A.



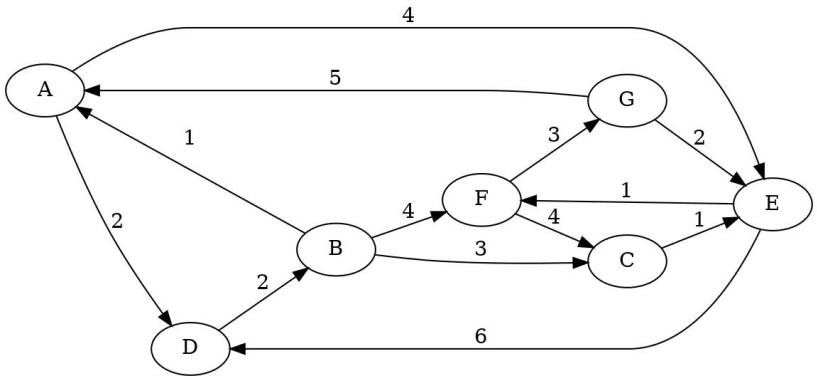
Indicate the total weight of the final minimum spanning tree. Report one single integer value. No other symbols must be included in the response. This is an example of the response format: 13

Solution

```
UN-directed matrix ... stated as such.
Weighted matrix ... stated as such.
Vertex list: [ 0] A [ 1] B [ 2] C [ 3] D [ 4] E [ 5] F [ 6] G [ 7] H
Adjacency Matrix:
 0 1 0 3 0 0 0 0
 1 0 6 0 0 0 0 3
 0 6 0 0 0 1 0 0
 3 0 0 0 4 3 0 0
 0 0 0 4 0 0 0 4
 0 0 1 3 0 0 0 0
 0 0 0 0 0 0 0 2
 0 3 0 0 4 0 2 0
### Graph MST (Kruskal and Prim)
Kruskal
Sorted array of edges: src dst weight:
                         A  B   1
                         C  F   1
                         G  H   2
                         A  D   3
                         B  H   3
                         D  F   3
                         D  E   4
                         E  H   4
                         B  C   6
MST (list of edges):
Edge [ 0]  A - [ 0]  B ==> weight =  1
Edge [ 2]  C - [ 2]  F ==> weight =  1
Edge [ 6]  G - [ 6]  H ==> weight =  2
Edge [ 0]  A - [ 0]  D ==> weight =  3
Edge [ 1]  B - [ 1]  H ==> weight =  3
Edge [ 3]  D - [ 3]  F ==> weight =  3
Edge [ 3]  D - [ 3]  E ==> weight =  4
Total tree weight                  = 17
Prim
MST (list of edges):
Edge [ 0]  A - [ 1]  B ==> weight =  1
Edge [ 0]  A - [ 3]  D ==> weight =  3
Edge [ 3]  D - [ 5]  F ==> weight =  3
Edge [ 5]  F - [ 2]  C ==> weight =  1
Edge [ 1]  B - [ 7]  H ==> weight =  3
Edge [ 7]  H - [ 6]  G ==> weight =  2
Edge [ 3]  D - [ 4]  E ==> weight =  4
Total tree weight                  = 17
```

Response
17

Exercise 10 (1.0 points)
Given the following directed and weighted graph, apply Dijkstra's algorithm to find all shortest paths connecting node A with all the other nodes. When necessary, consider nodes and edges in alphabetical order.



Report the shortest paths to all vertices. Please, indicate the shortest path to all vertices sorted in alphabetic order (i.e, display the shortest path for A B C D etc.). Report a sequence of integer values separated by one single space. No other symbols must be included in the response. This is an example of the response: 0 3 2 6 8 etc.

Solution

```
Directed matrix ... stated as such.
Weighted matrix ... stated as such.
Vertex list: [ 0] A [ 1] B [ 2] C [ 3] D [ 4] E [ 5] F [ 6] G
Adjacency Matrix:
 0 0 0 2 4 0 0
 1 0 3 0 0 4 0
 0 0 0 0 1 0 0
 0 2 0 0 0 0 0
 0 0 0 6 0 1 0
 0 0 4 0 0 0 3
 5 0 0 0 2 0 0
### Graph SSSP (Dijkstra and Bellman-Ford)
Dijkstra
Shortest paths from vertex [ 0] A
Node [ 0] A:  0 (pred=-1)
Node [ 3] D:  2 (pred= 0)
Node [ 1] B:  4 (pred= 3)
Node [ 4] E:  4 (pred= 0)
Node [ 5] F:  5 (pred= 4)
Node [ 2] C:  7 (pred= 1)
Node [ 6] G:  8 (pred= 5)
Bellman Ford
Iteration  0: A:0 B:4 C:9 D:2 E:4 F:5 G:8
Iteration  1: A:0 B:4 C:7 D:2 E:4 F:5 G:8
Iteration  2: A:0 B:4 C:7 D:2 E:4 F:5 G:8
Shortest paths from vertex [ 0] A
Node [ 0] A:  0 (pred=-1)
Node [ 1] B:  4 (pred= 3)
Node [ 2] C:  7 (pred= 1)
Node [ 3] D:  2 (pred= 0)
Node [ 4] E:  4 (pred= 0)
Node [ 5] F:  5 (pred= 4)
Node [ 6] G:  8 (pred= 5)
```

Response
0 4 7 2 4 5 8

Exercise 11 (2.0 points)
Analyze the following program and indicate the exact output generated.

```
void f (char *);
int main(void) {
  char *s = "this is a string";
```

```
  char *d;
  d = strdup (s);
  f (d);
  fprintf (stdout, "%s", d);
  return (1);
}
void f (char *s) {
  int i, j, flag;
  i = j = flag = 0;
  while (j < strlen (s)) {
    if (s[j]==' ') {
      j++;
      flag = 1;
    } else {
      if (flag==1) {
        s[i] = s[j] + ('A'-'a');
        flag = 0;
      } else {
        s[i] = s[j];
      }
      i++;
      j++;
    }
  }
  s[i]='\0';
  return;
}
```

Please, report the exact program output with no extra symbols.

Response

thisIsAString

Exercise 12 (2.0 points)

The following data structure `element_t` specifies elements of a bi-linked list, i.e., a list in which each element includes two pointers, one referencing the element on the right and one the element on the left. The function inserts elements after reading them from file.

```
typedef struct element_s {
  char *name;
  char id[MAX_C];
  char data[MAX_D];
  int salary;
  struct element_s *left;
  struct element_s *right;
} element_t;
element_t * readFile (element_t *head, char *fileIn) {
  FILE *input;
  char riga[MAX_R], name[MAX];
  element_t *tmpPtr;
  input=fopen(fileIn, "r");
  if (input==NULL){
    printf("Error opening file!\n");
    return (head);
  }
  while (fgets(riga, MAX_R, input)!=NULL){
    tmpPtr = (element_t *) malloc (sizeof (element_t));
    if (tmpPtr==NULL){
      printf("Allocation Error.\n");
      exit (EXIT_FAILURE);
    }
    sscanf(riga, "%s %s %s %d",
      name, tmpPtr->id, tmpPtr->data, &tmpPtr->salary);
    tmpPtr->name = (char *) malloc ((strlen(name)+1)*sizeof(char));
    sprintf (tmpPtr->name, "%s", name);
    tmpPtr->right = head;
    tmpPtr->left = NULL;
    if (head!=NULL) {
      head->left = tmpPtr;
    }
    head = tmpPtr;
  }
  fclose (input);
  return (head);
}
```

Indicates which ones of the following statements are correct. Note that incorrect answers imply a penalty in the final score.

1. The file can be read also with a statement like: `fscanf (input, "%s%s%s%d", ...)`.
2. The parameter `head` must be passed as `element_t **head` not with as `element_t *head`.
3. Function malloc could be more efficiently performed by calloc $(strlen(name) + 1, sizeof(char)$.

4. The instruction `head=tmpPtr` and `head− >left=tmpPtr` are inserted in the wrong order
5. The instructruction `head− >left = tmpPtr` must be executed only if `head`! =`NULL`.
6. The variable `tmpPtr− >name` can be read directly by the function sscanf instead of reading the variable name.

Response

1. TRUE.
2. FALSE.
3. FALSE.
4. TRUE.
5. TRUE.
6. FALSE.

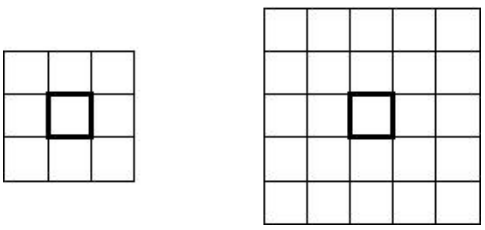
Exercise 13 (2.0 points)
Analyze the following recursive program.

```
int f (int n) {
  int m1, m2;
  if (n<=0) {
    return 0;
  }
  printf ("#");
  m1 = f (n-1);
  m2 = f (n-2);
  return m1+m2+1;
}
int main () {
  int rv;
  rv = f(5);
  fprintf (stdout, "%d", rv);
  return 1;
}
```

Report the exact program output with no other symbols.

Response
##########12

Exercise 14 (3.0 points)
Given a square matrix on integer values `mat` of size `n`, an integer value `k` (with $k < n$) we define the element in the neighboroud of tleemt `mat[r][c]` all those elements `mat[i][i]` such that $i \in [r-k, r+k]$ and $j \in [j-k, j+k]$.
The following picture represents neighborhoods of distances 1 and 2 with respect to the element in the center.

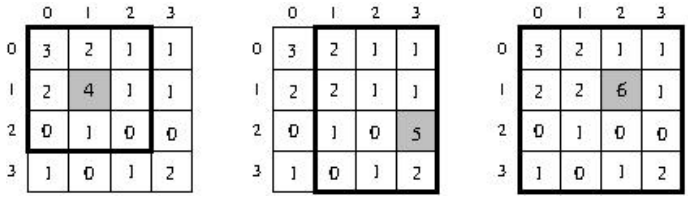


Write the function

```
void local_max (int **mat, int n, int k);
```

that displays all elements `mat[r][c]` and their coordinates `r` and `c` which are local maximum, i.e., are the largest value in their neighborhood of size `k`.
For example, in the matrices:

- On the left-hand side, the value 4 in `[1][1]` is a local maximum of degree 1 (and also 2).
- In the middle, the value 5 in `[2][3]` is a local maximum of degree 2 (and also 3).



- On the right-hand side, the value 6 in `[1][2]` is a local maximum of degree 3 (and also 1 and 2, obviously).

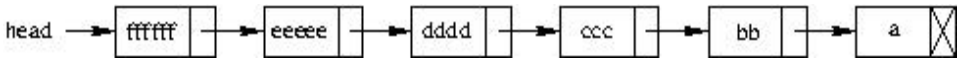
Write the entire program using standard C libraries but implement all required personal libraries. Modularize the program adequately, and report a brief description of the data structure and the logic adopted in plain English. Unclear or awkward programs, complex or impossible to understand, will be penalized in terms of the final evaluation.

Solution
`To be done.`

Exercise 15 (5.0 points)
A string includes sub-strings, made-up of alphabetic and numeric characters, and delimited by the full-stop "." character. Write the function

```
node_t *split_str (char *str);
```

which receives such a string as parameter `str` and it returns the pointer to a list in which each element includes a sub-string defined in a dynamic way. The candidate has to define the node structure of the list as well.
For example, if the function receives the string "a.bb.ccc.dddd.eeeee.fffff". is should create the following list: in which



all strings are dynamically defined, and it should return the pointer head to the list head.
Write the entire program using standard C libraries but implement all required personal libraries. Modularize the program adequately, and report a brief description of the data structure and the logic adopted in plain English. Unclear or awkward programs, complex or impossible to understand, will be penalized in terms of the final evaluation.

Solution
`To be done.`

Exercise 16 (9.0 points)
We are given an array of float values of size N. Each float value represents a cash flow in a bank account, i.e., a deposit (when the value is positive) or a withdrawal (when it is negative). The initial balance of the account is equal to zero. Given a sequence of cash flow operations, we define for each operation the current balance as the value resulting from the addition of the previous balance with the current flow operation. In this way, given a specific order for all cash flow operations, there is a maximum and a minimum current balance, even if the final balance is (obviously) always the same.
For example, with cash flow equal to (+10, -5, +7, -8) the balances are (0+10) +10, (10-5) +5, (5+7) +12, and (12-8) +4; thus the maximum and minimum balances are +12 and +4, with a difference equal to +8. At the same time, with the same set of cash flows but ordered as (-5, +10, -8, +7) the balances are -5, +5, -3, and +4, with a maximum and a minimum equal to +5 and -5 and a difference equal to +10.
Write a C program that, using a recursive algorithm, is able to find an order such that the difference between the maximum and the minimum balances is minimum. The prototype of the function is the following:

```
int *balance (float *flow, int n);
```

where flow is the initial array of integer cash flows and n is its size. The function has to return the pointer to the array storing the desired order of all n cash flows. Indicate (in plain English) which combinatory principles can be used when cash flows can have repeated values (e.g., (+10, -5, +10, +3)) and when cash flows are unique (all differ, e.g.

(+10, +5, -8, +3)). Can we apply different optimizations in the two cases? Indicate which combinatorics principle can be adopted in both cases and implement one of the two.
Write the entire program using standard C libraries but implement all required personal libraries. Modularize the program adequately, and report a brief description of the data structure and the logic adopted in plain English. Unclear or awkward programs, complex or impossible to understand, will be penalized in terms of the final evaluation.

Solution

`To be done.`